

Chapter 17: Guards

1. Overview

Guards are the router's authorization and flow-control checkpoints. Every navigation Angular's `Router` processes runs through a pipeline of matching, resolving, and — before the component tree is ever activated — a set of guard checks that decide **whether the navigation is allowed to proceed, be redirected, or be cancelled**.

Guards answer questions like:

- Is this user authenticated? (`CanActivate`)
- Is this user authenticated for *every* nested child route? (`CanActivateChild`)
- Does the user have unsaved changes and should we confirm before leaving? (`CanDeactivate`)
- Should this route even be considered a match, based on a feature flag or role? (`CanMatch`)
- Is the data safe to load, e.g., does this ID exist? (`CanLoad` — deprecated in favor of `CanMatch`)

Since Angular 14.2, guards are most idiomatically written as **functional guards** — plain functions using `inject()` — rather than injectable classes implementing marker interfaces. Class-based guards still work and are common in legacy codebases, but functional guards are now the default recommendation, are tree-shakeable, easier to compose, and eliminate boilerplate `@Injectable()` classes for what is often a two-line check.

Guards can return synchronously (`boolean`), asynchronously (`Observable<boolean|UrlTree>` or `Promise<boolean|UrlTree>`), or redirect (`UrlTree`). This chapter covers all guard types, functional and class-based forms, composition, execution order, and the `async/UrlTree` resolution semantics that interviewers probe most.

2. Core Concepts

2.1 The Guard Types

Guard	Interface / Function Type	Purpose	Runs When
<code>CanActivate</code>	<code>CanActivateFn</code> / <code>CanActivate</code>	Allow/deny entering a route	Before the route's component is instantiated
<code>CanActivateChild</code>	<code>CanActivateChildFn</code> / <code>CanActivateChild</code>	Allow/deny entering any child route	Before each child route activation
<code>CanDeactivate<T></code>	<code>CanDeactivateFn<T></code> / <code>CanDeactivate<T></code>	Allow/deny leaving a route (e.g., unsaved changes)	Before navigating away from the current component

Guard	Interface / Function Type	Purpose	Runs When
<code>CanMatch</code>	<code>CanMatchFn</code> / <code>CanMatch</code>	Decide if a route config should even be considered a match candidate	During route matching, before <code>CanActivate</code>
<code>Resolve<T></code>	<code>ResolveFn<T></code> / <code>Resolve<T></code>	Not strictly a guard, but part of the same pre-activation pipeline — pre-fetches data	After guards pass, before activation
<code>CanLoad</code> (deprecated)	<code>CanLoadFn</code> / <code>CanLoad</code>	Historically guarded lazy-loaded module loading	Superseded by <code>CanMatch</code> , which does both matching and load-prevention

Why `CanMatch` replaced `CanLoad`: `CanLoad` only ran once per app-lifetime the first time a lazily-loaded module's route was navigated to, and it prevented the router from trying alternate route configs if it failed (it just errored the whole navigation with no fallback). `CanMatch` runs on **every** navigation attempt to that path, and — critically — if it returns `false`, the router **continues trying other route definitions** that match the same path (enabling patterns like "same path, different route config based on role/flag").

2.2 Functional Guards with `inject()`

A functional guard is just a function conforming to a specific signature, registered directly in the route config:

```
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);
  return authService.isLoggedIn() ? true : router.createUrlTree(['/login']);
};
```

Key points:

- `inject()` must be called **synchronously within the guard function's execution context** (the router runs it inside an injection context). You cannot `await` something and then call `inject()` afterward — grab all dependencies first, then do async work.
- The function signature receives `route: ActivatedRouteSnapshot` and `state: RouterStateSnapshot` (for `CanActivate` / `CanActivateChild`), giving access to route params, query params, data, and the full future router state.
- Functional guards are plain functions — easily unit-tested by calling them inside `TestBed.runInInjectionContext(...)`.

2.3 Class-Based Guards (Legacy)

Pre-Angular 14.2 (and still valid), guards are injectable services implementing marker interfaces:

```

@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}

  canActivate(
    route: ActivatedRouteSnapshot,
    state: RouterStateSnapshot
  ): boolean | UrlTree {
    return this.auth.isLoggedIn() ? true : this.router.createUrlTree(['/login']);
  }
}

```

Registered in routes as `canActivate: [AuthGuard]` (the class, not an instance — Angular resolves it via DI).

Functional vs. class-based comparison:

Aspect	Functional	Class-based
Boilerplate	Minimal — a function	Requires <code>@Injectable</code> , class, constructor DI
Tree-shaking	Better — no always-provided service	Service instantiated even if guard logic is simple
Composability	Trivial to wrap/compose with helper HOFs	Requires inheritance or composition via injected services
Testing	Call directly or via <code>runInInjectionContext</code>	Standard Angular TestBed service testing
DI access	Via <code>inject()</code> inside the function	Via constructor injection
Angular version	14.2+	All versions

Angular's own schematics (`ng generate guard`) now default to generating functional guards.

2.4 Combining/Composing Guards

Multiple guards can be attached to the same route as an array:

```

{
  path: 'admin',
  canActivate: [authGuard, roleGuard],
  component: AdminComponent
}

```

All guards in the array must return truthy (or no `UrlTree` / `false`) for navigation to proceed. They run in array order, but conceptually behave like a logical AND — if any guard returns `false` or a `UrlTree`, the chain short-circuits and the remaining guards in that array are **not** invoked.

For OR-style composition (any-one-passes), you write a single functional guard that internally checks multiple conditions, or use a small composition helper:

```
export function orGuard(...guards: CanActivateFn[]): CanActivateFn {
  return (route, state) => {
    return combineLatest(
      guards.map(g => toObservable(runInInjectionContext(injector, () => g(route, state))))
    ).pipe(map(results => results.some(r => r === true)));
  };
}
```

(In practice, most teams just write the OR logic directly inside one guard rather than building generic combinators — simpler to read and debug.)

2.5 Guard Execution Order

For a single navigation, the router evaluates guards in this global order:

1. **CanDeactivate** of all currently-active components being left (deepest child first, moving up).
2. **CanMatch** for candidate routes (determines which route config, if any, matches — can cause the router to skip to the next candidate route).
3. **CanActivateChild** of all parent routes being entered, evaluated **top-down** (outermost parent first, then its children).
4. **CanActivate** of the route being entered, evaluated **top-down** as well — for a route with parents, parent **CanActivate** guards run before the child's own **CanActivate**.

So the full picture, top to bottom, for a navigation from Route A to nested Route B/C:

```
CanDeactivate (A, deepest active component first)
  ↓
CanMatch (candidate matching for B, then C)
  ↓
CanActivateChild (B's CanActivateChild, since C is B's child)
  ↓
CanActivate (B's CanActivate, then C's CanActivate)
  ↓
Resolvers (B's resolvers, then C's resolvers)
  ↓
Component instantiation
```

This ordering matters for interview questions: **CanDeactivate always runs before any CanActivate of the destination**, and **parent guards always run before child guards** on the way in, but **child (leaving) guards run before parent guards** on the way out — actually **CanDeactivate** only concerns the components literally being destroyed, evaluated deepest-first since you must confirm leaving the innermost view before its ancestors.

2.6 Async Guards — Observable / Promise / UrlTree

Guards may return:

- `boolean` — synchronous allow/deny.
- `UrlTree` — synchronous redirect (via `Router.createUrlTree()` or `Router.parseUrl()`).
- `Observable<boolean | UrlTree>` — the router subscribes and takes the **first emitted value**, then unsubscribes (equivalent to `first()` semantics). The observable must complete or emit at least once; an observable that never emits will hang the navigation forever.
- `Promise<boolean | UrlTree>` — awaited before proceeding.

```
export const rolesGuard: CanActivateFn = (route) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const required = route.data['roles'] as string[];

  return auth.currentUser$.pipe(
    take(1),
    map(user => (user && required.every(r => user.roles.includes(r)))
      ? true
      : router.createUrlTree(['/forbidden'])
    )
  );
};
```

Returning a `UrlTree` from any guard is the canonical, race-condition-safe way to redirect — **preferred over manually calling `router.navigate()` inside the guard and returning `false`**, because:

- A `UrlTree` redirect is treated as part of the *same* navigation resolution — the router replaces the target internally.
- Calling `router.navigate()` imperatively from inside a guard and then returning `false` triggers a **second, separate navigation**, which can race with the guard's own in-flight navigation, causing flicker, duplicate history entries, or inconsistent `NavigationCancel` events.

3. Code Examples

3.1 Functional Guards Using `inject()`

```
// auth.guard.ts
import { inject } from '@angular/core';
import { CanActivateFn, CanActivateChildFn, Router } from '@angular/router';
import { AuthService } from './auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) {
    return true;
  }

  // Preserve intended destination for post-login redirect
```

```

    return router.createUrlTree(['/login'], {
      queryParams: { returnUrl: state.url }
    });
  };

// Reuse the same logic for child routes so guests can't reach
// any nested admin path even if only the parent had the guard.
export const authChildGuard: CanActivateChildFn = (route, state) => authGuard(route, state);

```

```

// role.guard.ts – reads expected roles from route data
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth.service';

export const roleGuard: CanActivateFn = (route) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const allowedRoles = (route.data['roles'] as string[]) ?? [];

  const user = auth.getCurrentUser();
  if (!user) return router.createUrlTree(['/login']);

  const hasAccess = allowedRoles.length === 0 || allowedRoles.some(r =>
user.roles.includes(r));
  return hasAccess ? true : router.createUrlTree(['/forbidden']);
};

```

```

// app.routes.ts
export const routes: Routes = [
  {
    path: 'admin',
    canActivate: [authGuard, roleGuard],
    canActivateChild: [authChildGuard],
    data: { roles: ['ADMIN'] },
    children: [
      { path: 'users', component: UserListComponent },
      { path: 'settings', component: SettingsComponent, data: { roles: ['SUPER_ADMIN'] } }
    ]
  }
];

```

3.2 CanDeactivate — Unsaved Changes Confirmation

```
// can-deactivate.guard.ts
import { CanDeactivateFn } from '@angular/router';

export interface CanComponentDeactivate {
  canDeactivate: () => boolean | Observable<boolean>;
}

export const unsavedChangesGuard: CanDeactivateFn<CanComponentDeactivate> = (component) => {
  return component.canDeactivate ? component.canDeactivate() : true;
};
```

```
// edit-profile.component.ts
@Component({ /* ... */ })
export class EditProfileComponent implements CanComponentDeactivate {
  form = this.fb.group({ name: [''], email: [''] });

  canDeactivate(): boolean | Observable<boolean> {
    if (!this.form.dirty) {
      return true;
    }
    // Non-blocking confirm using a modal service that returns an Observable<boolean>
    return this.dialog
      .confirm('You have unsaved changes. Leave anyway?')
      .pipe(take(1));
  }
}
```

```
// route config
{
  path: 'profile/edit',
  component: EditProfileComponent,
  canDeactivate: [unsavedChangesGuard]
}
```

A raw `confirm()`-based synchronous version (simpler, but blocks the UI thread and can't be styled):

```
export const simpleUnsavedGuard: CanDeactivateFn<CanComponentDeactivate> = (component) => {
  if (component.canDeactivate && !component.canDeactivate()) {
    return confirm('Discard unsaved changes?');
  }
  return true;
};
```

3.3 CanMatch for Feature-Flagged Routes

```
// feature-flag.guard.ts
import { inject } from '@angular/core';
import { CanMatchFn, Route, UrlSegment, Router } from '@angular/router';
import { FeatureFlagService } from './feature-flag.service';

export const featureFlagGuard: CanMatchFn = (route: Route, segments: UrlSegment[]) => {
  const flags = inject(FeatureFlagService);
  const flagName = route.data?.['flag'] as string;
  return flags.isEnabled(flagName);
};
```

```
// app.routes.ts – same path, two competing configs based on flag
export const routes: Routes = [
  {
    path: 'dashboard',
    canMatch: [featureFlagGuard],
    data: { flag: 'newDashboard' },
    loadChildren: () => import('./dashboard-v2/dashboard-v2.component')
      .then(m => m.DashboardV2Component)
  },
  {
    path: 'dashboard',
    loadChildren: () => import('./dashboard-v1/dashboard.component')
      .then(m => m.DashboardComponent)
  }
];
```

If `featureFlagGuard` returns `false` for the first `dashboard` entry, the router does **not** error — it falls through and matches the second `dashboard` route. This is the key behavioral difference from `CanActivate` / `CanLoad`, and it's what makes `CanMatch` ideal for A/B tests, role-based route swapping, and progressive feature rollout without duplicating path segments in confusing ways.

`CanMatch` guards can also return a `UrlTree` to redirect immediately if no candidate should match:

```
export const requireBetaAccessGuard: CanMatchFn = () => {
  const flags = inject(FeatureFlagService);
  const router = inject(Router);
  return flags.isEnabled('betaProgram') ? true : router.createUrlTree(['/not-found']);
};
```

4. Internal Working

4.1 The Router's Pipeline

Internally, the Angular Router models navigation as an RxJS pipeline of operators applied to a `NavigationTransition` object, roughly:

NavigationTransition

- Recognize (match URL to route config, running CanMatch here)
- GuardsCheck (runs CanDeactivate → CanActivateChild → CanActivate, in that order)
- Resolve (runs Resolvers if guards passed)
- Activate (creates/destroys component instances via the router outlet)

Each of these is a distinct RxJS operator (`recognize`, `checkGuards` / `applyRedirects`, `resolveData`, `activateRoutes` in modern implementations) chained with `switchMap` / `mergeMap`, so a later navigation can cancel an in-flight one (`NavigationCancel` with `NavigationCancellationCode.SupersededByNewNavigation`).

4.2 How CanMatch Integrates with Recognition

`CanMatch` runs as part of **route recognition**, before the guard-check phase even begins, because whether a route "exists" for this URL is not yet decided. The recognizer:

1. Takes the ordered array of route configs at the current level.
2. For each candidate whose `path` matches the URL segment, it runs any `canMatch` guards.
3. If all pass (or none defined) → this route is selected, recognition recurses into its children.
4. If any `canMatch` guard returns `false` → the recognizer discards this candidate and tries the **next sibling route config** with the same or overlapping path.
5. If no candidate route matches at all → `NoMatch` is thrown, and (if a wildcard `**` exists or `enableTracing` / error handling is configured) resolves to a 404-style handling; otherwise the navigation fails silently or throws.
6. If a `canMatch` guard returns a `UrlTree` → recognition aborts and a redirect navigation begins immediately, similar to a guard-level redirect.

This is why `CanMatch` guards receive `(route: Route, segments: UrlSegment[])` — the *static config object*, not an `ActivatedRouteSnapshot` — because at this point there is no resolved route state yet, only the raw config and unconsumed URL segments.

4.3 The GuardsCheck Phase — Ordering Mechanics

Once recognition succeeds and a full future `RouterStateSnapshot` exists, the router computes:

- `canDeactivateChecks`: derived by diffing the *current* activated route tree against the *future* one. Any node present in current-but-not-in-future (or whose component type/route config identity changed) gets a `CanDeactivate` check. These are ordered **child-to-parent** (deepest first) because you conceptually leave the innermost view first.
- `canActivateChecks`: nodes present in future-but-not-in-current (or reused-but-config-changed). Ordered **parent-to-child** (outermost first) because you must be authorized for a section before being authorized for something nested inside it.

The router runs all `CanDeactivate` checks first (as an array of observables combined and checked sequentially/collectively), and only if **all** pass does it proceed to run `CanActivateChild` + `CanActivate` for the incoming tree, again parent-first.

Internally each individual guard result is normalized into an `Observable<boolean | UrlTree>` — a plain `boolean` is wrapped with `of()`, a `Promise` is wrapped with `from()`, and the pipeline applies `first()` (or equivalent single-value extraction) plus `every()`-like logic across the guard array so that a single `false/UrlTree` short-circuits (via `takeWhile` semantics) and skips invoking subsequent guards in the same phase.

4.4 Resolving `UrlTree` Redirects

When any guard (`CanMatch`, `CanActivate`, `CanActivateChild`) yields a `UrlTree` instead of `true/false`:

1. The router treats it as **"redirect requested."**
2. The current navigation transition is not simply continued; the router calls `router.navigateByUrl(urlTree)` (internally reusing the same transition machinery) which starts recognition over again from the new URL — the original navigation resolves with a `NavigationCancel` (redirect variant) and a fresh, related navigation begins.
3. `NavigationEnd` will ultimately fire for the *new* URL, not the originally requested one — meaning `Router.events` subscribers see `NavigationStart(original) → NavigationCancel/Redirect(original) → NavigationStart(redirect target) → ... → NavigationEnd(redirect target)`.
4. Because this is handled inside the router's own transition machinery (not a manual `.navigate()` call from guard code), there is no race condition and browser history is correctly managed (by default the redirect replaces rather than pushes, depending on `NavigationBehaviorOptions` you may pass to `createUrlTree`).

5. Edge Cases & Gotchas

- `inject()` **called after an `await`**: Illegal — `inject()` only works synchronously in the initial call stack of the guard function (or inside `runInInjectionContext`). Resolve all dependencies first, then do async work with them.

```
// WRONG
export const badGuard: CanActivateFn = async () => {
  await someAsyncSetup();
  const auth = inject(AuthService); // throws: no active injection context
  return auth.isLoggedIn();
};
```

- `CanDeactivate` **does not run on browser tab close / refresh**. For that you need the `beforeunload` window event, which supports only a native browser confirm dialog (no custom UI), and is entirely separate from Angular's router.
- `CanDeactivate` **is skipped if the component itself is never destroyed via router navigation** — e.g., programmatically destroying a component another way, or when `ChildrenOutletContexts` / manual DOM manipulation bypasses the router; the guard only fires on router-driven navigation transitions.
- **Guards on lazy-loaded feature module root routes**: applying `canActivate` on the parent route that owns `loadChildren` guards entry into the whole lazy module. If you want to actually prevent the **module chunk from being fetched at all**, you need `canMatch` (or the deprecated `canLoad`), not

`canActivate` — `canActivate` still triggers the dynamic `import()` before the guard result is checked in older setups; `canMatch` decides before recognition proceeds into that branch, preventing the module from loading.

- **Array order does not imply priority beyond short-circuiting.** `canActivate: [guardA, guardB]` — if `guardA` returns `false`, `guardB` never runs. This is often surprising when `guardB` has side effects (e.g., logging/analytics) that a developer expected to always fire.
- **Returning `false` vs `UrlTree`:** Returning `false` silently cancels navigation with a `NavigationCancel` event and the URL bar can flash/revert; there's no user-facing feedback unless you subscribe to router events. Prefer `UrlTree` redirects for anything user-facing.
- **Guards re-run on every navigation to the same route with different params**, e.g., `/users/1` → `/users/2` on a route with `runGuardsAndResolvers` default (`paramsChange`-ish behavior, actually default is `PathParamsChange`) — by default, guards only re-run if the *matched path itself* changes identity or configured triggers fire; fine-tune via the `runGuardsAndResolvers` route property (`always`, `paramsOrQueryParamsChange`, `paramsChange`, `pathParamsChange`, or a custom predicate function) if you need guards to re-fire on query param changes alone.
- **`CanActivateChild` applies to *all* descendant routes, not just the immediate child** — placing it on a top-level route guards every nested route beneath it, which is powerful but easy to forget when auditing "which guard protects this deeply nested route."
- **Observable guards that never complete/emit hang navigation forever** with no error and no timeout by default — always ensure guard observables use `take(1) / first()` or genuinely complete; a subject that's never `next()`-ed will freeze the app on that route transition.
- **Testing pitfall:** unit testing a functional guard requires an injection context — calling `authGuard(route, state)` directly outside Angular throws `NG0203`. Use:

```
TestBed.runInInjectionContext(() => authGuard(routeSnapshot, stateSnapshot));
```

- **Guard vs. Resolver ordering confusion:** guards run *before* resolvers. If a guard depends on data a resolver would fetch, you must fetch it yourself inside the guard (or share a cached service) — you cannot rely on resolver data being available to guards.
- **`CanMatch` failing on *all* candidates and no wildcard route** results in an unhandled `NoMatch` — many teams forget to add a catch-all `{ path: '**', component: NotFoundComponent }` specifically because `CanMatch` failures are a newer, less obvious way to hit "no route matched."

6. Interview Questions & Answers

Q1. What is the purpose of a route guard in Angular?

A guard is a function (or, historically, an injectable class) the router consults before allowing a navigation to activate a route, activate a child route, deactivate the current route, or even match a route config. Guards return `true / false`, a `UrlTree` (redirect), or an `Observable / Promise` of those, letting the router decide whether to proceed, cancel, or redirect.

Q2. What's the difference between `CanActivate` and `CanActivateChild`?

`CanActivate` protects entry into the specific route it's attached to. `CanActivateChild`, attached to a parent route, protects entry into **any** of that parent's descendant routes — so placing it once on a parent guards the whole subtree without repeating `canActivate` on every child.

Interviewer intent: checks whether the candidate understands that guard placement isn't 1:1 with the route being protected — a common source of "why didn't my guard fire" bugs.

Q3. Why would you use `CanMatch` instead of `CanActivate`?

`CanMatch` runs during route *recognition*, before a route config is even considered "matched." Unlike `CanActivate` (which assumes the route already matched and just checks authorization), a `false` from `CanMatch` lets the router discard that candidate and try the next route config with an overlapping path — enabling multiple route definitions for the same path selected by feature flag, role, or A/B bucket. It also prevents a lazy-loaded chunk from being fetched at all, unlike `CanActivate`.

Q4. What replaced `CanLoad`, and why is it deprecated?

`CanMatch` replaced `CanLoad`. `CanLoad` only prevented lazy-chunk loading and, on failure, hard-failed the navigation with no fallback route. `CanMatch` does everything `CanLoad` did (blocks loading of the lazy config) but also participates properly in route recognition, allowing multiple competing route definitions at the same path and returning `UrlTree` redirects.

Q5. How do you write a functional guard, and what's `inject()` doing there?

A functional guard is a plain function matching a router-defined type (e.g., `CanActivateFn = (route, state) => ...`), registered directly in the `Routes` array. `inject()` retrieves dependencies (services) from Angular's DI system using the *ambient injection context* the router sets up while invoking the guard — it's the same mechanism `inject()` uses inside field initializers of a class, just re-purposed for standalone functions. It must be called synchronously during the guard's initial execution; calling it after an `await` throws `NG0203` because the injection context is no longer active.

Q6. What can a guard return, and what does each return type mean?

- `true` / `false` (or a `Promise` / `Observable` of these) — allow or deny, synchronously or asynchronously.
- `UrlTree` — redirect to a different location; treated as part of the router's own navigation resolution, not a manual `navigate()` call.
- `Observable<boolean|UrlTree>` — router subscribes and takes the first emission (must complete or emit — an observable that never emits hangs the navigation).
- `Promise<boolean|UrlTree>` — awaited before the router proceeds.

Q7. Why is returning a `UrlTree` preferred over calling `router.navigate()` and returning `false`?

Returning a `UrlTree` lets the router fold the redirect into the *same* transition machinery — it internally re-triggers recognition against the new URL as a related, sequential step. Calling `router.navigate()` imperatively from inside the guard and separately returning `false` starts a **second, independent** navigation that can race with the original one's cancellation, potentially causing flicker, duplicate `NavigationCancel` / `NavigationStart` events, or incorrect browser history entries.

Interviewer intent: distinguishes candidates who've only seen guard tutorials from those who've debugged real navigation race conditions in production.

Q8. Explain the overall order guards execute in during a single navigation.

`CanDeactivate` (for components being left, deepest/innermost first) → `CanMatch` (for candidate routes being entered, during recognition) → `CanActivateChild` (top-down, outermost parent first) → `CanActivate` (top-down). Only after all of these pass do resolvers run, followed by component activation.

Q9. If a route has `canActivate: [GuardA, GuardB]`, and `GuardA` returns `false`, does `GuardB` run?

No. The array behaves like a logical AND with short-circuit evaluation — the moment one guard denies (returns `false` or a `UrlTree`), the router stops evaluating the remainder of that phase's guard array and cancels/redirects the navigation. This is a frequent gotcha for guards that also perform logging/analytics side effects, since those side effects silently stop firing once an earlier guard fails.

Q10. How would you implement an "unsaved changes" confirmation using `CanDeactivate`?

Define a `CanDeactivateFn<T>` that calls a method on the leaving component (e.g., `component.canDeactivate()`), where the component itself knows whether its form is dirty. If dirty, show a confirmation (ideally via an async dialog service returning `Observable<boolean>`, not a blocking `window.confirm`), and return that result. Attach it via `canDeactivate: [unsavedChangesGuard]` on the route. Key subtlety: `CanDeactivate` only fires on router-driven navigations — closing the browser tab requires a separate `window:beforeunload` listener.

Q11. Are functional guards and class-based guards interchangeable? Can you mix them on the same route?

Yes to both — `canActivate` accepts an array where each entry can be either a `CanActivateFn` (functional) or a class reference implementing `CanActivate` (Angular resolves it via DI). Angular normalizes both forms internally before invoking them, so `canActivate: [funcGuard, ClassGuard]` is legal, though for consistency most codebases standardize on one style (functional is now the default recommendation from the Angular team going forward, especially for standalone-component apps).

Q12. What happens, internally, when `CanMatch` fails for every candidate route matching a given path, and there's no wildcard route?

The router's recognizer cannot resolve the URL to any route config, throwing a `NoMatch`-style error internally. Since there's no `**` wildcard fallback configured, the navigation fails — typically surfacing as an unhandled error/`NavigationError` event rather than a friendly 404 page. This is why teams that adopt `CanMatch` for feature flags must always keep a catch-all wildcard route, something not as commonly forgotten with plain `CanActivate` since that always requires a route match to have already succeeded before the guard even runs.

Interviewer intent: verifies the candidate understands `CanMatch` operates at a structurally earlier phase than `CanActivate`, with different failure semantics (no match at all, vs. a matched-but-denied route).

Q13. How do you unit test a functional guard given it relies on `inject()`?

You can't call it as a bare function outside Angular's DI — doing so throws `NG0203 (inject() must be called from an injection context)`. Wrap the call in `TestBed.runInInjectionContext(() => authGuard(routeSnapshot, stateSnapshot))`, after configuring `TestBed` with the services the guard depends on (real or mocked via `TestBed.configureTestingModule({ providers: [...] })`).

Q14. Can a guard depend on data fetched by a resolver on the same route?

No — guards execute **before** resolvers in the pipeline (`GuardsCheck` phase precedes `Resolve` phase). If a guard needs data a resolver would normally fetch, the guard must fetch/cache it independently (e.g., via a shared service with an in-memory cache) rather than relying on `ActivatedRouteSnapshot.data` populated by that route's own resolver, since it won't be populated yet at guard-evaluation time.

Q15. What's `runGuardsAndResolvers` and why would you change it from its default?

It's a route config option controlling when guards/resolvers re-run for navigations that reuse the same route but change params/query params — options include `'paramsChange'`, `'pathParamsChange'` (default-ish behavior), `'paramsOrQueryParamsChange'`, `'always'`, or a custom predicate function (`from, to`) => `boolean`. You'd change it, e.g., to `'paramsOrQueryParamsChange'` or `'always'` if your `CanActivate`/resolver logic depends on query params (like a `tab` query param controlling which sub-view loads) and you need the guard to re-evaluate even though the path segment itself didn't change.

7. Quick Revision Cheat Sheet

- **Guard types:** `CanActivate` (enter route), `CanActivateChild` (enter any child route), `CanDeactivate` (leave route — unsaved changes), `CanMatch` (should this route config even match — feature flags, replaces deprecated `CanLoad`), `Resolve` (not a guard, but same pipeline phase, runs after guards).
- **Functional guard signature:** `CanActivateFn = (route: ActivatedRouteSnapshot, state: RouterStateSnapshot) => boolean | UrlTree | Observable<...> | Promise<...>`. Use `inject()` synchronously at the top, before any `await`.
- **Class-based guard:** `@Injectable({providedIn:'root'}) class X implements CanActivate { canActivate(route, state) {...} }`; register the class reference, not an instance.
- **Return values:** `true / false` = allow/deny; `UrlTree` = redirect (preferred over manual `router.navigate()` + `false`); `Observable / Promise` = async, first emission wins.
- **Array = AND with short-circuit:** `canActivate: [A, B]` — if `A` fails, `B` never runs.
- **Execution order:** `CanDeactivate` (deepest leaving component first) → `CanMatch` (route recognition) → `CanActivateChild` (parent→child) → `CanActivate` (parent→child) → Resolvers → Activation.
- **`CanMatch` vs `CanActivate`:** `CanMatch` runs during recognition and can fall through to the *next* route config with the same path (and blocks lazy-chunk loading); `CanActivate` assumes the route already matched and only allows/denies.
- **`UrlTree` redirects** are resolved inside the router's own transition machinery (`NavigationCancel` on original + new `NavigationStart / NavigationEnd` for target) — race-free, unlike imperative `router.navigate()` calls inside a guard.
- `CanDeactivate` **doesn't cover tab close/refresh** — use `window:beforeunload` for that (native browser dialog only).
- **Guards run before resolvers** — never assume resolver data is available inside a guard on the same route.
- `runGuardsAndResolvers` controls re-evaluation on param/query-param changes for the same matched route (`paramsChange`, `pathParamsChange`, `paramsOrQueryParamsChange`, `always`, or custom function).
- **Testing functional guards:** must run inside an injection context — `TestBed.runInInjectionContext(() => guardFn(route, state))`.
- **No wildcard + `CanMatch` failing everywhere** = unhandled `NoMatch / NavigationError`; always pair `CanMatch`-based feature flags with a `**` fallback route.